Detecção de Bad Smells e Refatoração Segura

Disciplina: Teste de Software

Trabalho: Detecção de Bad Smells e Refatoração Segura

Aluno: Breno de Oliveira Brandão

Matrícula: 813789

Repositório: github.com/brenin35/bad-smells-js-refactoring

1. Análise de Smells

Todo o código de produção está em um único método, `generateReport`, com 58 linhas. Ele monta o cabeçalho, filtra os itens, marca prioridades, formata as linhas, soma o total e monta o rodapé. A leitura manual encontrou três smells principais.

1.1 Método Longo (Long Method)

Como o método acumula cinco responsabilidades, é preciso ler as 58 linhas para entender qualquer uma delas, e uma mudança em uma parte pode quebrar outra. **Impacto nos testes:** não dá para testar a regra de filtragem ou o cálculo do total isoladamente. Todo teste precisa gerar o relatório inteiro e inspecionar a string final.

1.2 Condicionais repetidas e código duplicado

A condição `reportType === 'CSV'` / `'HTML'` aparece em quatro pontos: cabeçalho, linha do ADMIN, linha do USER e rodapé. Dentro do laço, ela ainda fica aninhada em `user.role === 'ADMIN'` / `'USER'`. O resultado é duplicação literal: o template da linha CSV está escrito duas vezes, e `total += item.value` aparece quatro vezes. Adicionar um formato novo, como PDF, exigiria editar quatro blocos diferentes (o smell que Fowler chama de *Shotgun Surgery*). **Impacto nos testes:** os caminhos crescem de forma combinatória (formatos × papéis × faixas de valor), e as cópias podem divergir sem que ninguém perceba.

1.3 Números e strings mágicos (Magic Numbers)

Os valores `1000` (limite de prioridade) e `500` (limite de visibilidade do usuário comum) aparecem sem nome, assim como `'ADMIN'` e `'USER'`. As regras de negócio só estão explicadas nos comentários. **Impacto nos testes:** para escrever casos de borda, como valor exatamente 500 ou 1000, é preciso descobrir os limites lendo o código. E uma mudança de regra exige caçar cada ocorrência.

Além desses três, o método tem um efeito colateral oculto: ele altera os objetos recebidos (`item.priority = true`). Por isso, os testes precisam clonar os dados de entrada com `JSON.parse(JSON.stringify(...))`.

2. Relatório da Ferramenta

O repositório já trazia um `.eslintrc.json` com `eslint:recommended` e `plugin:sonarjs/recommended`. Ele é mais rígido que o do enunciado: o limite de complexidade cognitiva é 5, e não 15, e também ativa `no-duplicated-branches`. Essa configuração foi mantida.

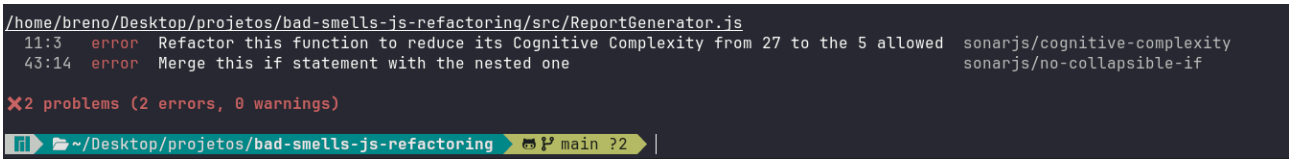


Figura 1: saída de `npx eslint src/` antes da refatoração.

A ferramenta complementou a análise manual em dois pontos:

- **Complexidade cognitiva quantificada (27).** A leitura manual diz que o método "é complexo". O SonarJS mede quanto cada `if`, `else` e laço contribui, com penalidade extra para o aninhamento, que chega a 4 níveis. O valor 27 passa até do limite de 15 do enunciado, e a impressão subjetiva vira um critério verificável.
- **If colapsável (linha 43).** A regra `no-collapsible-if` apontou que `else if (user.role === 'USER')` seguido de `if (item.value <= 500)` poderia ser uma condição só. É um detalhe estrutural fácil de deixar passar quando a atenção está nos smells maiores.

Por outro lado, a ferramenta não apontou os números mágicos, a duplicação dentro do mesmo método nem o efeito colateral. As regras de duplicação do SonarJS comparam funções e ramos idênticos, não trechos repetidos dentro de ramos diferentes. Ferramenta e análise manual são, portanto, complementares.

Houve também um problema de ambiente: `npm test` não rodava, porque o `package.json` usa `"type": "module"` e o Jest precisa da flag de ESM do Node. O script foi alterado para `node --experimental-vm-modules node_modules/jest/bin/jest.js`, sem mexer nos testes. Além disso, os testes do repositório ficam em `tests/`, e não em `__tests__/`, como diz o enunciado.

3. Processo de Refatoração

O smell mais crítico é a combinação de **Método Longo** com **condicionais repetidas** no laço principal, que concentra a maior parte da complexidade. A refatoração foi feita em `src/ReportGenerator.refactored.js`, rodando a suíte de testes a cada passo.

Antes (laço principal de `generateReport`):

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) {
      item.priority = true;
    }
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? 'style="font-weight:bold;"' : '';
      report += `<tr ${style}><td>${item.id}</td>...</tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        report += `${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
      } else if (reportType === 'HTML') {
        report += `<tr><td>${item.id}</td>...</tr>\n`;
        total += item.value;
      }
    }
  }
}
// + mais dois blocos if (reportType === ...) para cabeçalho e rodapé
```

Depois (métodos de `CsvFormatter` condensados em uma linha para caber na página):

```
const PRIORITY_THRESHOLD = 1000;
const USER_VALUE_LIMIT = 500;

const VISIBILITY_RULES = {
  ADMIN: () => true,
  USER: (item) => item.value <= USER_VALUE_LIMIT,
```

```
};

class CsvFormatter {
  header()        { return ['ID,NOME,VALOR,USUARIO']; }
  row(item, user) { return `${item.id},${item.name},${item.value},${user.name}`; }
  footer(total)   { return ['', 'Total,,', `${total},,`]; }
}
// HtmlFormatter implementa a mesma interface: header(), row() e footer()

const FORMATTERS = { CSV: new CsvFormatter(), HTML: new HtmlFormatter() };

generateReport(reportType, user, items) {
  const formatter = FORMATTERS[reportType];
  if (!formatter) {
    return '';
  }
  const visible = visibleItems(items, user);
  const lines = [
    ...formatter.header(user),
    ...visible.map((item) => formatter.row(item, user)),
    ...formatter.footer(sumValues(visible)),
  ];
  return lines.join('\n').trim();
}
```

Técnicas aplicadas, do catálogo de Fowler:

- **Replace Conditional with Polymorphism:** cada formato virou uma classe (`CsvFormatter` e `HtmlFormatter`) com a mesma interface. Os quatro `if (reportType ...)` viraram uma única busca no mapa `FORMATTERS`. Um formato novo agora é só uma classe nova, sem tocar em `generateReport`.
- **Extract Function:** `visibleItems` (filtragem por papel), `isPriority` (regra de destaque) e `sumValues` (total) ganharam nome próprio e podem ser testadas isoladamente.
- **Decompose Conditional:** o `if/else` por papel virou a tabela `VISIBILITY_RULES`, o que também eliminou o `if` colapsável.
- **Replace Magic Number with Symbolic Constant:** `PRIORITY_THRESHOLD` e `USER_VALUE_LIMIT`.
- **Replace Loop with Pipeline:** `filter`, `map` e `reduce` no lugar do laço com acumuladores. Isso eliminou as quatro cópias de `total += item.value` e a mutação dos itens recebidos.

4. Resultados e Rede de Segurança

O teste do arquivo refatorado, `tests/ReportGenerator.refactored.test.js`, é uma cópia exata do original, com apenas o `import` alterado. Depois da refatoração, `npx eslint src/ReportGenerator.refactored.js` não reporta nenhum problema, mesmo com o limite de complexidade 5.

Métrica	Original	Refatorado
Complexidade cognitiva de `generateReport`	27	abaixo de 5
Erros do ESLint	2	0
Pontos que testam `reportType`	4	1 (busca no mapa)
Ocorrências de `total += item.value`	4	0 (`sumValues`)
Testes passando	4 de 5	5 de 5



Figura 2: saída de `npm test -- --verbose` com as duas suítes (original e refatorada).

A falha no arquivo original não foi introduzida pela refatoração. É um defeito que já existia: para um ADMIN, as linhas HTML sem prioridade saem como `<tr >` (com espaço), mas o teste espera `<tr>`. Isso acontece porque o espaço fica fixo no template, e o atributo `style` vazio não o remove. Na versão refatorada, o espaço faz parte do próprio atributo (`' style="..."'`), e os cinco testes passam. Uma comparação automatizada entre as saídas dos dois arquivos confirmou que essa é a única diferença de comportamento. Ela cobriu os formatos CSV, HTML e desconhecido, os papéis ADMIN, USER e outro, e listas cheias e vazias.

5. Conclusão

A suíte de testes foi o que tornou a refatoração segura. Ela verifica o comportamento (a string gerada) e não a implementação, então foi possível reescrever toda a estrutura interna, trocando condicionais por polimorfismo e laços por pipelines, e confirmar a cada passo que nada mudou para quem usa a classe. Ela ainda revelou um defeito do código original que passaria despercebido em uma leitura.

A redução dos Bad Smells melhorou o código de forma mensurável. A complexidade cognitiva caiu de 27 para menos de 5, cada regra de negócio tem agora um nome e um único lugar, e adicionar um formato ou um papel de usuário deixou de exigir mudanças espalhadas. Código mais simples é mais fácil de ler, de testar por partes e de modificar sem medo. É essa confiança, sustentada pelos testes, que permite ao design continuar evoluindo.